

Week 2 Project

Build Your RAG Application

Submit your project here: <https://forms.gle/1EYiudaGDfp2eY9f8>

Have questions? Email tanish@thegenacademy.com

What this week is about

This week you are building a RAG application using one of the available Week 2 tracks: LangChain/LangGraph, n8n, Lyzr or any other frameworks you might want to explore. You will pick a corpus, chunk it, embed it, store it, retrieve from it, and generate cited answers.

Most RAG projects do not fail at the model. They fail at chunking, retrieval quality, or evaluation. The framework on the next page forces a decision at each layer of the stack so you do not skip the parts that matter.

How this project works

You will make two independent choices.

Choice	Option 1	Option 2
Your use case	Bring your own. Use the framework on page 2 to scope a RAG problem you actually care about.	Pick one of four suggested use cases (page 5). Each comes pre-scoped using the framework.
Your build track	No-code with n8n or Lyzr. Use n8n or Lyzr for visual workflows	Code-heavy with LangChain + LangGraph. Python primitives plus stateful graph flows.

Part 1: The RAG Framework

The Primer: Your one-liner

Before the framework, write a single sentence that captures your whole RAG app. If you cannot say it in one line, you do not yet know what you are retrieving, for whom, or how good it has to be.

My RAG app helps [USER] answer [QUESTION TYPE] from [KNOWLEDGE CORPUS] in [SURFACE] with [%] faithfulness and/or [%] relevance.

Worked example

My RAG app helps new hires and employees answer benefits and policy questions from the company HR handbook in Slack with 95% faithfulness.

Three rules for the one-liner

Name the corpus specifically. "Our docs" is a corpus of. "~100 HR policy pages in Confluence updated quarterly" is. The corpus drives every chunking and retrieval decision downstream.

Faithfulness, not just relevance. Your success metric should measure whether the answer is grounded in the retrieved docs, not just whether it sounds reasonable. Set a target percentage.

Latency is a first-class constraint. RAG pipelines stack up time fast (retrieval + reranking + generation). Pick a ceiling now so you do not over-engineer retrieval and ship something that takes 30 seconds.

The Framework

Fill out every field in 1 to 2 sentences. The framework forces a decision at every layer of the RAG stack so nothing gets hand-waved.

Field	Fill in (1 to 2 sentences max)
Use case (one line)	What question are you answering, who asks it, and where does it show up (Slack, widget, API, internal agent)?
Corpus	Sources, rough doc count, formats, language, and who owns the source of truth.
Ingestion + cleaning	How docs get in, and what cleaning happens before chunking (strip markup, drop boilerplate, decode entities)
Ingestion + freshness	How docs get in, refresh cadence, freshness SLA.
Chunking + embedding	Chunk strategy and size, semantic vs fixed, embedding model, and why.
Retrieve	Store, dense / sparse / hybrid, and top-k.

Tips before you fill it in

Pick chunk size and embedding model together. A 512-token chunk on a 384-dim embedding is wasted; a 2000-token chunk on a small model loses signal. Match capacity.

Hybrid retrieval is usually right. Pure dense misses exact matches (error codes, names, ticker symbols). Pure BM25 misses semantic intent. Combine them and re-rank.

Your "I don't know" path matters more than your happy path. A RAG app that hallucinates when retrieval fails is worse than one that says "I could not find this in our docs." Design the refusal first.

Part 2: Pick Your Build Track

The framework is identical across tracks. Your track determines how you implement it. Pick by Tuesday so you have four full days to build.

How to decide

Default to Track 2 (LangChain + LangGraph) if you write code regularly. You will learn more about what is actually happening under the hood, and it generalizes to production systems.

Pick Track 1 (n8n or Lyrzr) if you are non-technical, time-constrained, or your use case fits an available no-code path.

Quick note: You'll have access to Nebius credits, which you can use for model calls, including both embeddings and generation.

Track 1: No-code with n8n or Lyrz

What it is

A no-code path where you build RAG without writing Python. In n8n, you wire AI Agent, Embeddings, Vector Store, and Document Loader nodes together. In Lyrz, you configure the supported agent workflow for the Financial Document Intelligence or Customer Support Bot project.

Best for

Rapid prototyping, integrations-heavy workflows (CRM, helpdesk, Slack), demos for non-technical stakeholders, and builders who want a guided no-code path.

Key building blocks

n8n: AI Agent node, Embeddings nodes (OpenAI, Cohere, HuggingFace), Vector Store nodes (Pinecone, Qdrant, Supabase), Document Loaders, prompt templates, Webhook triggers.

Lyrz: configured agent workflow, project-specific knowledge base setup, prompt instructions, and evaluation examples. Lyrz credits have been provided to you.

Tradeoffs

Less control over chunking, reranking, and custom evals than code. You will trade depth for speed and guided setup.

Track 2: Code-heavy with LangChain + LangGraph

What it is

Python framework with low-level RAG primitives (loaders, splitters, embeddings, retrievers, chains) plus LangGraph for stateful, multi-step graph flows.

Best for

Production-grade systems, custom retrieval logic (parent-doc, self-query, multi-query), complex multi-hop or agentic RAG, anyone writing evals as code.

Key building blocks

DocumentLoaders, TextSplitters, Embeddings, VectorStores, Retrievers, LCEL chains, LangGraph state machines, LangSmith for tracing and evals.

Tradeoffs

Steeper ramp, more code to maintain, more decisions to make explicit. The upside is full control and a portfolio piece you can extend.

Part 3: Suggested Use Cases

Each use case below is a fully filled framework. Pick one, adapt the user and corpus to your context, and you have a Week 2 scope. The core four are intentionally varied across corpus type and retrieval pattern so you can match what you want to learn. Lyrz no-code variants are now available for Financial Document Intelligence Pipeline RAG and Customer Support Bot.

#	Use case	Retrieval pattern	Best for
1	Enterprise Policy Q&A Bot	Hybrid + rerank	HR, IT, ops teams
2	Financial Document Intelligence	Table-aware hybrid	Investment analysts
3	Graph RAG for Org Knowledge	Graph + vector	PMs, cross-team ICs
4	Customer Support KB	Hybrid + multilingual	Support agents, customers

Details about each use-case:

Project 1: Enterprise Policy Q&A Bot

Description

Build a RAG-powered Q&A system over real enterprise documents — HR policies, compliance manuals, product documentation, or onboarding guides. Use a no-code/low-code tool or a guided RAG starter (e.g., Pinecone + LangChain template) to ingest your documents, chunk and embed them, and stand up a question-answering interface. Then stress-test it with 15 questions including edge cases: ambiguous queries, questions that span multiple documents, and questions the knowledge base simply can't answer. Document where retrieval succeeds, where it fails, and why.

Best For

All profiles — PMs use product docs, finance analysts use regulatory filings, consultants use client knowledge bases, engineers use technical documentation. Everyone has documents they wish they could query.

Deliverable

Working Q&A bot + a 15-question evaluation report with retrieval quality scores and failure analysis.

Submission

Demo recording + GitHub link or zip file + evaluation report document.

Difficulty

Beginner to Intermediate | No-code option available | Engineers extend with custom embeddings and reranking

Project 2: Financial Document Intelligence Pipeline RAG

Description

Build a RAG pipeline that answers questions across financial documents - SEC filings, earnings call transcripts, insurance claims, or loan documents. Implement two chunking strategies (fixed-size vs. semantic chunking) and compare retrieval quality on the same set of queries. Add a reranking step and measure the improvement. This project is particularly relevant for anyone working in financial services, insurance, or regulated industries, but the techniques apply to any domain with dense, structured documents. This project now has code, n8n, and Lyzr tracks in the Week 2 project list.

Best For

Data Scientists/Analysts (financial analysis), Finance roles (Wells Fargo, JPMC, Citi, Morgan Stanley, NYLife), Architects (pipeline design), Consultants (client financial analysis).

Deliverable

A working financial RAG pipeline with a chunking strategy comparison report and reranking impact analysis.

Submission

Demo recording + GitHub link or zip file + comparison report document.

Difficulty

Intermediate | Code, n8n, and Lyzr options | PMs can define test questions and evaluate business relevance of retrieved answers

Project 3: GraphRAG for Organizational Knowledge

Description

Model your team's or organization's knowledge as a graph — people, projects, skills, documents, and decisions — and use GraphRAG to enable queries that traditional vector search can't handle well. Think: "Who worked on the last compliance audit and what tools did they use?" or "What decisions were made about our pricing model and who approved them?" Build the graph with 20+ nodes, run queries against it, and compare GraphRAG results with traditional vector-based RAG on the same questions. This project highlights when structured relationships matter more than semantic similarity.

Best For

Program/Project Managers (organizational knowledge), Executives (decision tracking), Consultants (client engagement knowledge), Tech Leads (technical decision logs), Strategy roles.

Deliverable

A knowledge graph with 20+ nodes + GraphRAG vs. vector RAG comparison on 10 queries + analysis of when each approach wins.

Submission

Demo recording + GitHub link or zip file + comparison analysis document.

Difficulty

Advanced | Code required | PMs/Managers can design the graph schema and evaluate results

Project 4: Customer Support Knowledge Base with Hybrid Search

Description

Build a customer support bot that combines keyword search and semantic search (hybrid retrieval) over support tickets, FAQs, and product manuals. Implement a confidence-based fallback: when the system is not sure of an answer, it escalates to a human rather than hallucinating. Test with 20 real-world-style support queries and measure first-contact resolution rate. Highly relevant for e-commerce, SaaS, and consumer-facing companies where bad AI answers directly cost customer trust. The Lyzr Customer Support Bot variant gives no-code builders a guided version of this project.

Best For

Product Managers (customer experience), Software Engineers (implementation), Founders (MVP for their product), QA Engineers (testing edge cases), Commerce/Retail roles.

Deliverable

Working support bot with hybrid search + escalation logic, tested against 20 queries with resolution metrics.

Submission

Demo recording + GitHub link or zip file + evaluation metrics document.

Difficulty

Intermediate | Code, n8n, and Lyrz options | Engineers add custom reranking

Optional Add-ons - Bonus Points

Add-on: Chatbot UI (Vibe Coded)

Description

Vibe code a chatbot and connect it as the front-end interface for your Week 2 RAG system. Your chatbot now has a real knowledge base behind it — turning a simple conversational UI into a grounded, document-backed assistant. This is how the weekly projects start compounding.

How to submit

Deliverables for Week 2

Project documentation	Submit a Google Doc explaining what you built. Include: project overview, datasets used, prompts or agent instructions you used, iterations you tried, and any learnings or observations from the workflow.
Video demo	Submit a video (5 minutes or less) where you walk through your application, explain what you built, describe how you used AI coding tools, and demonstrate the final result live.
Project assets	Upload your code, workflow JSON, or Lyrz project assets to GitHub and share a link in the form below

** Solutions for use-cases

Given below are some solutions that we have put together for the use cases shared in this particular week's suggested use case section (part 3).

We highly encourage you **NOT** to look at this before you get started with your project. We intentionally don't want you to go through this because it will direct your thinking in a particular direction.

We would rather want you to think through the solution and build this yourself, even if it takes you a little bit more time. Only refer to the following documents if you are absolutely stuck and are unable to make progress. Use them as a hint document rather than replicating the following solutions. If you end up replicating the following solutions, you will not be given scores.

PS: The folder below for each use case contains the code for LangChain, JSON/workflow files, and setup files. It has the detailed solution doc as well as a demo video.

#	Use case	n8n	LangChain/LangGraph
1	Enterprise Policy Q&A Bot	 n8n	 Langchain and La...
2	Financial Document Intelligence	 n8n	 Langchain and La...
3	Graph RAG for Org Knowledge	 n8n	 Langchain and La...
4	Customer Support KB	 n8n	 Langchain and La...

Lyzr solutions

#	Use case	Lyzr
1	Financial Document Intelligence	 Lyzr
2	Customer Support KB	 Lyzr